Universidad Nacional Autónoma de México
Facultad de Ciencias



Inteligencia Artificial

Laboratorio 2.1

Docente: *Dra. JessicaSarahiMéndezRincón*

Alumna: *OsorioLopezClaudiaYarinet*

Fecha de realización: 14/09/2026

BFS

BFS es fundamental no solo para grafos, sino como base para agentes que buscan metas en espacios de estados (como resolver un cubo Rubik o encontrar la ruta más corta en un mapa sin pesos). BFS garantiza encontrar el camino más corto en grafos donde todas las aristas valen lo mismo. En inteligencia artificial, esto se conoce como un algoritmo de búsqueda completo (siempre encuentra la solución si existe) y óptimo.

```
from collections import deque
```

```
def bfs(grafo, inicio):
    # Conjunto para rastrear nodos visitados
    visitados = set()
    # Cola para manejar el orden de exploración (FIFO)
    cola = deque([inicio])

    # Marcamos el inicio como visitado
    visitados.add(inicio)

    print(f"Orden de exploración empezando desde {inicio}:")

    while cola:
        # Extraemos el nodo que llegó primero
        nodo_actual = cola.popleft()
        print(nodo_actual, end=" ")

        # Revisamos los vecinos del nodo actual
        for vecino in grafo[nodo_actual]:
            if vecino not in visitados:
                visitados.add(vecino)
                cola.append(vecino)
```

```
# Representación del grafo mediante un diccionario (Lista de adyacencia)
grafo_ejemplo = {
    0: [1, 2],
    1: [2],
    2: [0, 3],
    3: [3]
}
```

```
# Ejecución
bfs(grafo_ejemplo, 2)
```

```
Orden de exploración empezando desde 2:
2 0 3 1
```

DFS

```
def dfs(grafo, nodo, visitados=None):
    if visitados is None:
        visitados = set()

    # Marcamos el nodo como visitado y lo procesamos
    visitados.add(nodo)
    print(nodo, end=" ")

    # Exploramos recursivamente cada vecino
    for vecino in grafo[nodo]:
        if vecino not in visitados:
            dfs(grafo, vecino, visitados)

# Grafo de ejemplo
grafo_ejemplo = {
    0: [1, 2],
    1: [2],
    2: [0, 3],
    3: [3]
}

print("Orden de exploración DFS empezando desde 2:")
dfs(grafo_ejemplo, 2)
```

```
Orden de exploración DFS empezando desde 2:
2 0 1 3
```

Costo Uniforme

```
import heapq

def ucs(grafo, inicio, meta):
    # Cola de prioridad: (costo_acumulado, nodo_actual, camino)
    pq = [(0, inicio, [inicio])]
    visitados = {}

    while pq:
        (costo, nodo, camino) = heapq.heappop(pq)

        if nodo not in visitados or costo < visitados[nodo]:
            visitados[nodo] = costo

            if nodo == meta:
                return camino, costo

            for vecino, peso in grafo.get(nodo, []):
                nuevo_costo = costo + peso
                heapq.heappush(pq, (nuevo_costo, vecino, camino + [vecino]))

    return None, float('inf')

# Grafo como lista de adyacencia con pesos: {nodo: [(vecino, peso), ...]}
grafo_pesado = {
    'A': [('B', 5), ('C', 1)],
    'B': [('D', 1)],
    'C': [('B', 2), ('D', 5)],
    'D': []
}

camino, costo = ucs(grafo_pesado, 'A', 'D')
print(f"Mejor camino: {camino} con costo total: {costo}")
```

```
Mejor camino: ['A', 'C', 'B', 'D'] con costo total: 4
```

DLS - Profundidad limitada

```
def dls(grafo, nodo, meta, limite, visitados=None):
    if visitados is None:
        visitados = set()

    print(f"Visitando {nodo} (Límite restante: {limite})")

    if nodo == meta:
        return [nodo]

    if limite <= 0:
        return "CORTE"

    visitados.add(nodo)
    corte_ocurrido = False

    for vecino in grafo.get(nodo, []):
        if vecino not in visitados:
            resultado = dls(grafo, vecino, meta, limite - 1, visitados.copy())

            if resultado == "CORTE":
                corte_ocurrido = True
            elif resultado is not None:
                return [nodo] + resultado

    return "CORTE" if corte_ocurrido else None

# Grafo de ejemplo
grafo = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'E': ['G']
}

# Buscar 'G' con límite 2 (No debería encontrarlo, está a nivel 3)
resultado = dls(grafo, 'A', 'G', 2)
print(f"\nResultado con L=2: {resultado}")

# Buscar 'G' con límite 3 (Debería encontrarlo)
resultado = dls(grafo, 'A', 'G', 3)
print(f"Resultado con L=3: {resultado}")
```

```
Visitando A (Límite restante: 2)
Visitando B (Límite restante: 1)
Visitando D (Límite restante: 0)
Visitando E (Límite restante: 0)
Visitando C (Límite restante: 1)
Visitando F (Límite restante: 0)

Resultado con L=2: CORTE
Visitando A (Límite restante: 3)
Visitando B (Límite restante: 2)
Visitando D (Límite restante: 1)
Visitando E (Límite restante: 1)
Visitando G (Límite restante: 0)
Resultado con L=3: ['A', 'B', 'E', 'G']
```

Primero el mejor

```
import heapq

def greedy_best_first(grafo, heuristicas, inicio, meta):
    # La prioridad es solo la heurística h(n)
    pq = [(heuristicas[inicio], inicio, [inicio])]
    visitados = set()

    while pq:
        h_val, nodo, camino = heapq.heappop(pq)
```

```
        if nodo == meta:
            return camino

        if nodo not in visitados:
            visitados.add(nodo)

            for vecino in grafo.get(nodo, []):
                if vecino not in visitados:
                    heapq.heappush(pq, (heuristicas[vecino],
                                        vecino, camino + [vecino]))
    return None

# Grafo y Heurísticas
grafo = {'A': ['B', 'C'], 'B': ['D'], 'C': ['D'], 'D': []}
h = {'A': 10, 'B': 8, 'C': 2, 'D': 0}

print(f"Camino Greedy: {greedy_best_first(grafo, h, 'A', 'D')}")
```

```
Camino Greedy: ['A', 'C', 'D']
```

A*

```
import heapq

def a_estrella(grafo, heuristicas, inicio, meta):
    # pq guarda: (f_costo, g_costo, nodo_actual, camino)
    pq = [(heuristicas[inicio], 0, inicio, [inicio])]
    visitados = {}

    while pq:
        f, g, nodo, camino = heapq.heappop(pq)

        if nodo == meta:
            return camino, g

        if nodo not in visitados or g < visitados[nodo]:
            visitados[nodo] = g

            for vecino, costo_arista in grafo.get(nodo, []):
                nuevo_g = g + costo_arista
                nuevo_f = nuevo_g + heuristicas.get(vecino, 0)
                heapq.heappush(pq, (nuevo_f, nuevo_g, vecino, camino + [vecino]))

    return None, float('inf')

# Datos: Grafo (pesos reales) y Heurísticas (estimaciones)
grafo = {
    'A': [('B', 2), ('C', 4)],
    'B': [('D', 5)],
    'C': [('D', 1)],
    'D': []
}
h = {'A': 6, 'B': 4, 'C': 1, 'D': 0}

camino_optimo, costo_total = a_estrella(grafo, h, 'A', 'D')
print(f"Camino A*: {camino_optimo} | Costo Real: {costo_total}")
```

```
Camino A*: ['A', 'C', 'D'] | Costo Real: 5
```

A* Pesado

```
import heapq

def a_estrella_pesado(grafo, h, inicio, meta, w=2.0):
    # pq: (f_costo, g_costo, nodo_actual, camino)
    pq = [(w * h[inicio], 0, inicio, [inicio])]
    visitados = {}

    nodos_expandidos = 0

    while pq:
```

```
        f, g, nodo, camino = heapq.heappop(pq)
        nodos_expandidos += 1

        if nodo == meta:
            return camino, g, nodos_expandidos

        if nodo not in visitados or g < visitados[nodo]:
            visitados[nodo] = g

            for vecino, costo_arista in grafo.get(nodo, []):
                nuevo_g = g + costo_arista
                # Aplicamos el PESO w a la heurística
                nuevo_f = nuevo_g + (w * h.get(vecino, 0))
                heapq.heappush(pq, (nuevo_f, nuevo_g, vecino, camino + [vecino]))

    return None, float('inf'), nodos_expandidos

# Datos de prueba
grafo = {'A': [('B', 10), ('C', 1)], 'C': [('D', 10)], 'B': [('D', 1)], 'D': []}
h = {'A': 10, 'B': 1, 'C': 5, 'D': 0}

# Comparación rápida
camino, costo, exp = a_estrella_pesado(grafo, h, 'A', 'D', w=1.0)
print(f"A* Estándar (w=1): Costo {costo}, Nodos {exp}")

camino_p, costo_p, exp_p = a_estrella_pesado(grafo, h, 'A', 'D', w=5.0)
print(f"A* Pesado (w=5):   Costo {costo_p}, Nodos {exp_p}")
```

```
A* Estándar (w=1): Costo 11, Nodos 4
A* Pesado (w=5):   Costo 11, Nodos 3
```

CSP Simple (Backtracking)

```
def es_seguro(variable, valor, asignacion, restricciones):
    for vecino, regla in restricciones.get(variable, []):
        if vecino in asignacion and not regla(valor, asignacion[vecino]):
            return False
    return True

def backtracking_csp(variables, dominios, restricciones, asignacion={}):
    if len(asignacion) == len(variables):
        return asignacion # ¡Solución encontrada!

    var = [v for v in variables if v not in asignacion][0]

    for valor in dominios[var]:
        if es_seguro(var, valor, asignacion, restricciones):
            asignacion[var] = valor
            resultado = backtracking_csp(variables, dominios,
                                         restricciones, asignacion)
            if resultado: return resultado
            del asignacion[var] # Backtrack

    return None

# Ejemplo: Colorear A y B con colores distintos
vars = ['A', 'B']
doms = {'A': ['Rojo', 'Azul'], 'B': ['Rojo', 'Azul']}
restrs = {'A': [('B', lambda a, b: a != b)], 'B': [('A', lambda b, a: b != a)]}

print(f"Solución CSP: {backtracking_csp(vars, doms, restrs)}")
```

```
Solución CSP: {'A': 'Rojo', 'B': 'Azul'}
```

Minimax Simple (Gato/Tic-Tac-Toe)

```
import math

# Representamos el tablero como una lista de 9 espacios
# 1 para 'X' (MAX), -1 para 'O' (MIN), 0 para vacío
tablero_test = [1, -1, 1,
                0, -1, 0,
```

```
                0,  0, 0]

def evaluar(t):
    # Combinaciones ganadoras
    vias = [[0,1,2],[3,4,5],[6,7,8],[0,3,6],[1,4,7],[2,5,8],[0,4,8],[2,4,6]]
    for v in vias:
        if t[v[0]] == t[v[1]] == t[v[2]] != 0:
            return 10 if t[v[0]] == 1 else -10
    return 0

def movimientos_posibles(t):
    return [i for i, x in enumerate(t) if x == 0]

def aplicar(t, m, jugador):
    nuevo_t = t.copy()
    nuevo_t[m] = jugador
    return nuevo_t

def minimax(tablero, profundidad, es_max):
    puntuacion = evaluar(tablero)

    # Casos base: Alguien ganó o empate (profundidad máxima o no hay movimientos)
    if puntuacion == 10 or puntuacion == -10:
        return puntuacion
    if not movimientos_posibles(tablero):
        return 0

    if es_max:
        mejor = -float('inf')
        for m in movimientos_posibles(tablero):
            mejor = max(mejor, minimax(aplicar(tablero, m, 1), profundidad + 1, False))
        return mejor
    else:
        mejor = float('inf')
        for m in movimientos_posibles(tablero):
            mejor = min(mejor, minimax(aplicar(tablero, m, -1), profundidad + 1, True))
        return mejor

# --- PRUEBA ---
print("Calculando el mejor valor para el tablero actual...")
resultado = minimax(tablero_test, 0, False) # Es turno de MIN ('O')
print(f"Resultado del Minimax: {resultado}")
```

```
Calculando el mejor valor para el tablero actual...
Resultado del Minimax: -10
```

Cómo interpretar los resultados

Cuando ejecutas el código, el valor de retorno te dice el futuro del juego si ambos juegan perfecto:

- Si devuelve 10: Significa que, sin importar lo que haga el oponente, el jugador MAX tiene una estrategia para ganar.
- Si devuelve -10: Significa que el jugador MIN puede forzar una victoria.
- Si devuelve 0: El juego terminará en empate si nadie comete errores.

El paso final: ¿Cuál es el mejor movimiento?

El código que tienes solo te dice qué tan bueno es el estado actual, pero no te dice dónde mover. Para eso, necesitas una función envolvente (wrapper) que elija el índice:

```
def elegir_mejor_movimiento(tablero):
    mejor_valor = -float('inf')
    movimiento_final = -1

    for m in movimientos_posibles(tablero):
        # Simulamos el movimiento y vemos qué dice minimax
        valor = minimax(aplicar(tablero, m, 1), 0, False)
        if valor > mejor_valor:
            mejor_valor = valor
            movimiento_final = m

    return movimiento_final

print(f"La IA recomienda mover a la posición: {elegir_mejor_movimiento(tablero_test)}")
```

```
    La IA recomienda mover a la posición: 7
```

Si intentas probar esto en un juego más complejo (como el Ajedrez), notarás que el programa se queda "colgado". Esto pasa porque el árbol de búsqueda es demasiado grande. Para esos casos, es obligatorio implementar la Poda Alpha-Beta que mencionamos antes para "cortar" ramas innecesarias

---

© 2026 UNAM Facultado de Ciencias – Todos los derechos reservados